OpenFOAM Simulation Guide

Hydrogen–Methane T-Junction DDES
Equivalent Setup to ANSYS Fluent

Yash

April 18, 2026

Contents

1 Introduction **3**

1.1 Simulation Overview . . . 3

1.2 Fluent-to-OpenFOAM Equivalence . . . 3

1.3 Case Directory Structure . . . 4

2 Geometry and Meshing **5**

2.1 Geometry Definition . . . 5

2.2 Step 1: Generate the STL . . . 5

2.3 Step 2: Background Mesh (`blockMesh`) . . . 6

2.4 Step 3: Feature Extraction . . . 6

2.5 Step 4: `snappyHexMesh` . . . 6

2.5.1 Castellated Mesh . . . 6

2.5.2 Snapping . . . 7

2.5.3 Boundary Layers . . . 7

2.5.4 Boundary Patches . . . 7

3 Physical Models and Turbulence **8**

3.1 Turbulence Model (`momentumTransport`) . . . 8

3.1.1 Key Settings . . . 8

3.1.2 Turbulent Schmidt Number . . . 9

3.2 Species Transport (`thermophysicalProperties`) . . . 9

3.2.1 Thermo-type Breakdown . . . 10

3.2.2 Species Properties (NIST Reference Data) . . . 10

3.3 Chemistry and Combustion (Disabled) . . . 10

3.4 Gravity . . . 11

4 Boundary Conditions **12**

4.1 Velocity (`0/U`) . . . 12

4.1.1 Branch Velocity Derivation . . . 12

4.2 Pressure (`0/p`) . . . 13

4.3 Temperature (`0/T`) . . . 13

4.4 Species Mass Fractions (`0/H2`, `0/CH4`) . . . 13

4.5 Turbulence Fields (`0/nuTilda`, `0/nut`) . . . 14

4.6 Thermal Diffusivity (`0/alphat`) . . . 14

5 Solution Methods and Controls **15**
5.1 Pressure–Velocity Coupling (`fvSolution`) . . . 15
5.1.1 Linear Solvers . . . 15
5.2 Discretisation Schemes (`fvSchemes`) . . . 15
5.2.1 Temporal . . . 15
5.2.2 Gradient . . . 16
5.2.3 Convection (Divergence) . . . 16
5.2.4 Laplacian and Surface-Normal Gradient . . . 16
5.3 Under-Relaxation Factors . . . 17

6 Run Setup, Monitors, and Execution **18**
6.1 Step 1: Time-Stepping Parameters . . . 18
6.2 Step 2: Monitors (Function Objects) . . . 18
6.2.1 Residuals . . . 18
6.2.2 Surface Monitors . . . 19
6.2.3 Point Monitors (Wall H_2 Probes) . . . 19
6.3 Step 3: Autosave . . . 19
6.4 Step 4: Initialisation . . . 20
6.4.1 Default Fields . . . 20
6.4.2 Patch Branch Region (`setFields`) . . . 20
6.5 Step 5: Decompose and Run . . . 20
6.5.1 Domain Decomposition . . . 20
6.5.2 Run the Solver . . . 21
6.5.3 Reconstruct . . . 21
6.6 Step 6: During the Run — What to Watch . . . 21
6.7 Step 7: Enable Time Statistics (After $t = 4\,\mathrm{s}$) . . . 21

7 Post-Processing **22**
7.1 Hydrogen Volume Fraction . . . 22
7.2 Visualisation in ParaView . . . 22
7.3 Coefficient of Variation (COV) . . . 23
7.4 Monitoring Data . . . 23

8 Quick-Start Summary **24**

Chapter 1

Introduction

This guide documents the complete OpenFOAM setup for a transient Delayed Detached Eddy Simulation (DDES) of hydrogen injection into a methane-filled T-junction pipeline. Every dictionary file, boundary condition, and solver setting is a direct translation of the validated ANSYS Fluent configuration described in the companion *Parameterised CFD Simulation Guide*.

1.1 Simulation Overview

- **Physics:** Compressible binary-species mixing (H_2 / CH_4), no combustion.
- **Turbulence:** Spalart–Allmaras DDES with curvature correction.
- **Operating pressure:** 6.9 MPa.
- **Solver:** `reactingFoam` (compressible, multi-species, transient).
- **Time step:** $\Delta t = 2 \times 10^{-4}$ s, 25 000 steps (5 s physical time).

1.2 Fluent-to-OpenFOAM Equivalence

Table 1.1 summarises the top-level mapping between the Fluent and OpenFOAM configurations.

Table 1.1: Fluent ↔ OpenFOAM equivalence summary.

Setting	**Fluent**	**OpenFOAM**
Solver	Pressure-Based, Transient	`reactingFoam`
Turbulence	SA-DDES	`SpalartAllmarasDDES`
P–V coupling	PISO	PIMPLE (`nOuterCorrectors 1`)
Momentum scheme	Bounded Central Differencing	LUST
Transient scheme	Bounded Second Order Implicit	`backward`
Species	Species Transport (no reactions)	`reactingMixture`, chemistry off
Meshing	ANSYS Meshing / Workbench	`blockMesh` + `snappyHexMesh`

1.3 Case Directory Structure

The complete case follows the standard OpenFOAM three-directory layout:

```
openfoam_case/
  0/                    Initial & boundary conditions
    U  p  T  H2  CH4  nuTilda  nut  alphat
  constant/             Physical properties & mesh
    g  momentumTransport  turbulenceProperties
    thermophysicalProperties  chemistryProperties
    combustionProperties  reactions
    triSurface/         STL geometry files
  system/               Solver & mesh controls
    controlDict  fvSchemes  fvSolution
    blockMeshDict  snappyHexMeshDict
    surfaceFeatureExtractDict  setFieldsDict
    decomposeParDict
  Allrun                Full workflow script
  Allclean              Clean script
  generateSTL.py        Geometry generator
```

Chapter 2

Geometry and Meshing

2.1 Geometry Definition

The T-junction consists of two intersecting cylinders:

Table 2.1: Geometry dimensions.

Component	Parameter	Value
Main pipe	Diameter d_1	460 mm
	Length	9200 mm
	Axis	Z-direction
Branch pipe	Diameter d_2	115 mm ($\beta = 0.25$)
	Length	1380 mm ($\geq 10\, d_2$)
	Axis	$+Y$-direction
	Junction location	$Z = 4600$ mm
	Injection angle	90°

2.2 Step 1: Generate the STL

A Python script (`generateSTL.py`) creates the T-junction surface as four separate STL files in `constant/triSurface/`:

1. `wall.stl` — combined wall surfaces of both pipes.
2. `main_inlet.stl` — cap at $Z = 0$.
3. `outlet.stl` — cap at $Z = 9.2$ m.
4. `branch_inlet.stl` — cap at the branch tip.

Run:

```
python3 generateSTL.py
```

Dependency: `numpy` and `numpy-stl` (`pip install numpy-stl`).

2.3 Step 2: Background Mesh (`blockMesh`)

The `blockMeshDict` creates a single hexahedral block enclosing the entire domain:

Table 2.2: `blockMeshDict` extents.

Direction	Min (m)	Max (m)
X	-0.30	$+0.30$
Y	-0.30	$+1.50$
Z	-0.10	$+9.30$

Cell count: $30 \times 90 \times 470$ (background cell size $\approx 20\,\text{mm}$).

```
blockMesh
```

2.4 Step 3: Feature Extraction

Extract sharp edges from the STL files so that `snappyHexMesh` can snap mesh vertices accurately to geometric features:

```
surfaceFeatureExtract
```

This reads `surfaceFeatureExtractDict` and produces `.eMesh` files for each STL surface.

2.5 Step 4: `snappyHexMesh`

The `snappyHexMeshDict` performs three operations:

2.5.1 Castellated Mesh

- Refines cells intersecting the STL surfaces to **level 2–3**.
- Applies **level 3** refinement inside a sphere of radius 690 mm centred at the junction $(0, 0, 4.6)$ — matching the Fluent BOI region ($1.5 \times d_1$).
- Removes cells outside the pipe geometry.
- `locationInMesh`: $(0, 0, 2.0)$ — a point guaranteed to be inside the main pipe.

2.5.2 Snapping

Mesh vertices are iteratively moved onto the STL surface. The feature edges (`.eMesh` files) ensure clean resolution of the pipe–branch intersection.

2.5.3 Boundary Layers

Five prismatic layers are added to the `wall` patch:

Table 2.3: Layer insertion parameters.

Parameter	Value
Surface layers	5
Expansion ratio	1.2
Final layer thickness	0.3 (relative)
Min thickness	0.1 (relative)
Feature angle	60°

```
snappyHexMesh -overwrite
```

The `-overwrite` flag writes the final mesh directly into `constant/polyMesh/` rather than creating time directories.

2.5.4 Boundary Patches

`snappyHexMesh` automatically creates the following patches from the named STL regions:

Table 2.4: Boundary patches.

Patch name	Type	Description
`main_inlet`	`patch`	Main pipe inlet ($Z = 0$)
`branch_inlet`	`patch`	Branch pipe inlet
`outlet`	`patch`	Main pipe outlet ($Z = 9.2\,\mathrm{m}$)
`wall`	`wall`	All pipe walls

Chapter 3

Physical Models and Turbulence

3.1 Turbulence Model (`momentumTransport`)

The turbulence model is configured in `constant/momentumTransport`:

constant/momentumTransport

```
simulationType  LES;

LES
{
    LESModel        SpalartAllmarasDDES;
    turbulence      on;
    printCoeffs     on;
    delta           cubeRootVol;

    cubeRootVolCoeffs
    {
        deltaCoeff  1;
    }

    SpalartAllmarasDDESCoeffs
    {
        curvatureCorrection yes;
    }
}
```

3.1.1 Key Settings

- **simulationType LES:** In OpenFOAM, DES/DDES models are classified under the LES framework. The `SpalartAllmarasDDES` model automatically switches between RANS behaviour in attached boundary layers and LES behaviour in separated regions.

- **Curvature correction:** Enabled to improve prediction accuracy in the curved flow region near the T-junction. Equivalent to the "Curvature Correction" checkbox in Fluent's viscous model panel.
- **delta cubeRootVol:** The LES filter width is taken as the cube root of the cell volume ($\Delta = V^{1/3}$), which is the standard choice for unstructured meshes.

3.1.2 Turbulent Schmidt Number

The turbulent Schmidt number is set in `constant/turbulenceProperties`:

constant/turbulenceProperties

```
Sct     0.5;
```

This matches the Fluent TUI command `/define/models/viscous/turbulent-schmidt-number 0.5`.

3.2 Species Transport (`thermophysicalProperties`)

The binary H_2/CH_4 mixture is defined in `constant/thermophysicalProperties`:

constant/thermophysicalProperties (header)

```
thermoType
{
    type            heRhoThermo;
    mixture         reactingMixture;
    transport       sutherland;
    thermo          janaf;
    energy          sensibleEnthalpy;
    equationOfState perfectGas;
    specie          specie;
}

inertSpecie CH4;
species ( H2 CH4 );
```

3.2.1 Thermo-type Breakdown

Table 3.1: `thermoType` entries and their Fluent equivalents.

Entry	OpenFOAM	Fluent Equivalent
`type`	`heRhoThermo`	Pressure-based energy with density
`mixture`	`reactingMixture`	Species Transport model
`transport`	`sutherland`	Kinetic Theory / Sutherland viscosity
`thermo`	`janaf`	Polynomial c_p (piecewise)
`equationOfState`	`perfectGas`	Ideal Gas density law
`energy`	`sensibleEnthalpy`	Energy Equation enabled

3.2.2 Species Properties (NIST Reference Data)

Each species entry contains molecular weight, JANAF thermodynamic coefficients, and Sutherland transport coefficients fitted to NIST data at 6.9 MPa:

Table 3.2: Species properties (NIST verification values at 284 K, 6.9 MPa).

Property	CH_4	H_2	Source
Molecular weight ($\mathrm{kg\,kmol^{-1}}$)	16.043	2.016	NIST
c_p ($\mathrm{J\,kg^{-1}\,K^{-1}}$)	2580	14 500	NIST
Thermal conductivity ($\mathrm{W\,m^{-1}\,K^{-1}}$)	0.038	0.185	NIST
Dynamic viscosity (Pa s)	1.25×10^{-5}	9.0×10^{-6}	NIST

3.3 Chemistry and Combustion (Disabled)

Since this is a mechanical mixing study (no combustion), the chemistry and combustion models are explicitly disabled:

constant/chemistryProperties

```
chemistryType
{
    chemistrySolver none;
}
chemistry       off;
```

constant/combustionProperties

```
combustionModel none;
```

constant/reactions

```
species ( H2 CH4 );
reactions ();
```

This is equivalent to setting **Reactions: Disabled** in Fluent's Species Transport panel.

3.4 Gravity

Gravity is set in `constant/g`:

constant/g

```
dimensions  [0 1 -2 0 0 0 0];
value       (0 -9.81 0);
```

Combined with the `perfectGas` equation of state, the solver fully resolves buoyancy-driven stratification (density ratio CH_4:H_2 ≈ 8 at 6.9 MPa). This is equivalent to Fluent's gravity enablement with *Specified Operating Density = 0*.

Chapter 4

Boundary Conditions

All boundary condition files reside in the `0/` directory. Each file specifies an `internalField` (initial value) and a `boundaryField` with entries for every patch.

4.1 Velocity (`0/U`)

Table 4.1: Velocity boundary conditions.

Patch	Type	Value	Notes
`main_inlet`	`fixedValue`	$(0,\ 0,\ 10)\,\mathrm{m/s}$	$10\,\mathrm{m/s}$ along $+Z$
`branch_inlet`	`fixedValue`	$(0,\ -32,\ 0)\,\mathrm{m/s}$	$32\,\mathrm{m/s}$ along $-Y$ (into main pipe)
`outlet`	`inletOutlet`	`inletValue` $(0, 0, 0)$	Prevents backflow
`wall`	`noSlip`	—	Stationary, no-slip

4.1.1 Branch Velocity Derivation

The branch velocity of $32\,\mathrm{m/s}$ is derived from the volumetric dilution factor $\mathcal{D} = 0.2$:

$$\mathcal{D} = \beta^2 \frac{u_2}{u_1} \quad \Rightarrow \quad 0.2 = (0.25)^2 \times \frac{u_2}{10} \quad \Rightarrow \quad u_2 = 32\ \mathrm{m/s}$$

4.2 Pressure (`0/p`)

Table 4.2: Pressure boundary conditions.

Patch	Type	Value (Pa)	Notes
`main_inlet`	`zeroGradient`	—	Extrapolated from interior
`branch_inlet`	`zeroGradient`	—	Extrapolated from interior
`outlet`	`fixedValue`	6 900 000	Operating pressure (gauge 0 relative)
`wall`	`fixedFluxPressure`	—	Adjusts pressure gradient for body forces

Note: In OpenFOAM's compressible solvers, `p` is the absolute pressure (not gauge). The `fixedValue` at the outlet is therefore set to the full operating pressure of 6.9 MPa. This is equivalent to Fluent's gauge pressure of 0 Pa with operating pressure 6.9 MPa.

4.3 Temperature (`0/T`)

Table 4.3: Temperature boundary conditions.

Patch	Type	Value (K)
`main_inlet`	`fixedValue`	284
`branch_inlet`	`fixedValue`	293
`outlet`	`inletOutlet`	284 (backflow)
`wall`	`zeroGradient`	— (adiabatic)

4.4 Species Mass Fractions (`0/H2`, `0/CH4`)

Table 4.4: H_2 mass fraction boundary conditions.

Patch	Type	Y_{H_2}
`main_inlet`	`fixedValue`	0 (pure CH_4)
`branch_inlet`	`fixedValue`	1 (pure H_2)
`outlet`	`inletOutlet`	0 (backflow)
`wall`	`zeroGradient`	—

CH_4 is the **inert (balance) species**: $Y_{CH_4} = 1 - Y_{H_2}$. Its boundary values mirror H_2 accordingly.

4.5 Turbulence Fields (`0/nuTilda`, `0/nut`)

Table 4.5: Turbulence boundary conditions.

Patch	Type	Value	Notes
`nuTilda` (SA working variable)			
`main_inlet`	`fixedValue`	8×10^{-7}	$\approx 3\nu$
`branch_inlet`	`fixedValue`	8×10^{-7}	
`outlet`	`inletOutlet`	8×10^{-7}	
`wall`	`fixedValue`	0	
`nut` (turbulent viscosity)			
Inlets / outlet	`calculated`	10^{-6}	Derived from `nuTilda`
`wall`	`nutUSpaldingWallFunction`	0	All-y^+ wall function

4.6 Thermal Diffusivity (`0/alphat`)

Table 4.6: Turbulent thermal diffusivity BCs.

Patch	Type
Inlets / outlet	`calculated`
`wall`	`compressible::alphatWallFunction` ($Pr_t = 0.85$)

Chapter 5

Solution Methods and Controls

5.1 Pressure–Velocity Coupling (`fvSolution`)

The PIMPLE algorithm with `nOuterCorrectors = 1` is used, which reduces to the PISO algorithm — matching the Fluent setup exactly.

system/fvSolution (PIMPLE block)

```
PIMPLE
{
    nOuterCorrectors    1;      // = PISO
    nCorrectors         2;
    nNonOrthogonalCorrectors 1;
    momentumPredictor   yes;
}
```

5.1.1 Linear Solvers

Table 5.1: Linear solver configuration.

Field	Solver	Preconditioner	Tolerance
`p`	GAMG	Gauss-Seidel	10^{-6}
`U, h, nuTilda`	PBiCGStab	DILU	10^{-6}
`H2, CH4`	PBiCGStab	DILU	10^{-8}

5.2 Discretisation Schemes (`fvSchemes`)

5.2.1 Temporal

```
ddtSchemes { default  backward; }
```

`backward` is second-order implicit and unbounded. It is the OpenFOAM equivalent of Fluent's **Bounded Second Order Implicit** transient formulation.

5.2.2 Gradient

```
gradSchemes { default  Gauss linear; }
```

Equivalent to Fluent's **Least Squares Cell Based** gradient.

5.2.3 Convection (Divergence)

Table 5.2: Convection scheme mapping.

Term	OpenFOAM Scheme	Fluent Equivalent
`div(phi,U)`	`Gauss LUST grad(U)`	Bounded Central Differencing
`div(phi,H2)` etc.	`Gauss limitedLinear 1`	Second Order Upwind
`div(phi,nuTilda)`	`Gauss limitedLinear 1`	Second Order Upwind
`div(phi,h)`	`Gauss limitedLinear 1`	Second Order Upwind

Why LUST for momentum? LUST (Linear-Upwind Stabilised Transport) is a blend of 75 % linear (central) and 25 % upwind. It provides the low numerical dissipation needed for resolving turbulent structures in DES regions while maintaining stability. This is the closest OpenFOAM equivalent to Fluent's Bounded Central Differencing.

5.2.4 Laplacian and Surface-Normal Gradient

```
laplacianSchemes { default  Gauss linear corrected; }
snGradSchemes    { default  corrected; }
```

The `corrected` scheme provides second-order accuracy on non-orthogonal meshes. This handles the equivalent of Fluent's **PRESTO!** pressure interpolation via the corrected Laplacian.

5.3 Under-Relaxation Factors

Table 5.3: Under-relaxation factors (matching Fluent).

Variable	**Factor**
Pressure (`p`)	0.3
Momentum (`U`)	0.7
Modified turbulent viscosity (`nuTilda`)	0.8
Species (`H2`, `CH4`)	0.9
Energy (`h`)	1.0
Density (`rho`)	1.0

Chapter 6

Run Setup, Monitors, and Execution

6.1 Step 1: Time-Stepping Parameters

Set in `system/controlDict`:

Table 6.1: Run calculation parameters.

Parameter	Value
Time step size, Δt	2×10^{-4} s
End time	5 s (= 25 000 steps)
Write interval	0.1 s (= 500 steps)
Write format	binary

The time step gives CFL ≈ 1.3 based on the branch velocity (32 m/s) and a representative cell size of ~ 5 mm in the junction region.

6.2 Step 2: Monitors (Function Objects)

All monitors are defined as function objects in the `functions` sub-dictionary of `controlDict`.

6.2.1 Residuals

Residual monitor

```
residuals
{
    type            solverInfo;
    libs            (utilityFunctionObjects);
    writeControl    timeStep;
    writeInterval   1;
    fields          (U p h nuTilda H2);
}
```

Output is written to `postProcessing/residuals/`.

6.2.2 Surface Monitors

Three surface monitors track mixing at the outlet and mass conservation:

Table 6.2: Surface monitors.

Name	Quantity	Patch	Operation
`outletH2`	H_2 mass fraction	`outlet`	Weighted average (`phi`)
`outletMassFlow`	Mass flow rate	`outlet`	Sum (`phi`)
`mainInletMassFlow`	Mass flow rate	`main_inlet`	Sum (`phi`)

Expected outlet H_2 mass fraction $\rightarrow\sim 0.149$ for $\mathcal{D} = 0.2$.

6.2.3 Point Monitors (Wall H_2 Probes)

Four probes monitor H_2 mass fraction at the pipe-top wall ($y = +0.23\,\mathrm{m}$) at downstream distances of $z/d_1 = 2, 4, 6, 8$ from the junction:

Table 6.3: Probe locations ($d_1 = 0.46\,\mathrm{m}$, junction at $Z = 4.6\,\mathrm{m}$).

Name	X (m)	Y (m)	Z (m)	z/d_1
`wall_h2_z2d`	0	0.23	5.52	2
`wall_h2_z4d`	0	0.23	6.44	4
`wall_h2_z6d`	0	0.23	7.36	6
`wall_h2_z8d`	0	0.23	8.28	8

Output is written to `postProcessing/wallH2Probes/`.

6.3 Step 3: Autosave

OpenFOAM's `writeInterval 0.1` (with `writeControl adjustableRunTime`) saves case data every 0.1 s of simulation time, equivalent to Fluent's autosave every 500 time steps.

6.4 Step 4: Initialisation

6.4.1 Default Fields

The `internalField` entries in each `0/` file provide the baseline initialisation (pure CH_4 at 284 K flowing at 10 m/s along $+Z$).

6.4.2 Patch Branch Region (`setFields`)

The `setFieldsDict` patches a cylindrical region around the branch with pure H_2 and the branch velocity:

system/setFieldsDict (region)

```
cylinderToCell
{
    p1      (0 0.20 4.6);
    p2      (0 1.40 4.6);
    radius  0.06;

    fieldValues
    (
        volScalarFieldValue H2  1
        volScalarFieldValue CH4 0
        volVectorFieldValue U   (0 -32 0)
    );
}
```

This is equivalent to Fluent's **Hybrid Initialisation** followed by **Patch** on the branch cell register.

```
setFields
```

6.5 Step 5: Decompose and Run

6.5.1 Domain Decomposition

```
decomposePar
```

The `decomposeParDict` uses `scotch` partitioning with 8 subdomains (matching the Fluent setup of 8 solver processes).

6.5.2 Run the Solver

```
mpirun -np 8 reactingFoam -parallel
```

Or simply use the provided `Allrun` script which executes the full workflow automatically:

```
./Allrun
```

6.5.3 Reconstruct

After the run completes, reconstruct the decomposed data for post-processing:

```
reconstructPar
```

6.6 Step 6: During the Run — What to Watch

1. **Residuals:** Check `postProcessing/residuals/` — all residuals should drop by at least 1–2 orders of magnitude within each time step.
2. **Mass balance:** Compare inlet and outlet mass flow rates. Imbalance $> 1\,\%$ indicates a problem.
3. **Outlet H_2:** Should stabilise near ~ 0.149 for $\mathcal{D} = 0.2$.
4. **Courant number:** The solver prints the maximum Courant number each time step. Target: $\mathrm{Co} < 2$. If consistently higher, reduce Δt to $1 \times 10^{-4}\,\mathrm{s}$.

6.7 Step 7: Enable Time Statistics (After $t = 4\,\mathrm{s}$)

The `fieldAverage1` function object in `controlDict` is pre-configured to begin time-averaging at $t = 4\,\mathrm{s}$ (`timeStart 4.0`). The first 20 000 steps (4 s) are treated as the initial transient; the final 5 000 steps (1 s) are used for time-averaged statistics.

Fields averaged:

- `UMean`, `UPrime2Mean` — mean velocity and Reynolds stresses.
- `pMean` — mean pressure.
- `H2Mean`, `H2Prime2Mean` — mean H_2 concentration and its variance (for COV calculation).
- `TMean` — mean temperature.

Chapter 7

Post-Processing

7.1 Hydrogen Volume Fraction

Fluent solves for mass fraction $Y_{\mathrm{H_2}}$, while the reference paper reports volume fraction $\phi_{\mathrm{H_2}}$. The conversion is:

$$\phi_{\mathrm{H_2}} = \frac{Y_{\mathrm{H_2}}/M_{\mathrm{H_2}}}{Y_{\mathrm{H_2}}/M_{\mathrm{H_2}} + (1 - Y_{\mathrm{H_2}})/M_{\mathrm{CH_4}}} \tag{7.1}$$

where $M_{\mathrm{H_2}} = 2.016\,\mathrm{kg/kmol}$ and $M_{\mathrm{CH_4}} = 16.043\,\mathrm{kg/kmol}$.

In OpenFOAM, compute this in ParaView using a **Calculator** filter:

```
(H2/2.016) / (H2/2.016 + (1-H2)/16.043)
```

Alternatively, add a `coded` function object to `controlDict` for automatic computation during the run.

7.2 Visualisation in ParaView

Open the reconstructed case:

```
paraFoam
```

Key visualisations:

1. **Contour plots:** H_2 mass/volume fraction on cross-sectional planes downstream of the junction.
2. **Streamlines:** Coloured by H_2 to visualise mixing patterns.
3. **Time-averaged fields:** Use `H2Mean` for time-averaged concentration, `UMean` for mean velocity.
4. **Wall y^+:** Plot `yPlus` (requires the `yPlus` function object or post-processing utility).

7.3 Coefficient of Variation (COV)

The COV quantifies mixing uniformity on a cross-section:

$$\mathrm{COV} = \frac{\sigma_\phi}{\bar{\phi}}$$

Compute from time-averaged fields:

- $\bar{\phi}$ from `H2Mean`.
- σ_ϕ from $\sqrt{\texttt{H2Prime2Mean}}$.

COV = 0 indicates perfect mixing; COV = 1 indicates complete segregation.

7.4 Monitoring Data

All function-object output is in `postProcessing/`:

Table 7.1: Post-processing output directories.

Directory	Contents
`residuals/`	Solver residual history
`outletH2/`	Mass-weighted H_2 at outlet vs. time
`outletMassFlow/`	Outlet mass flow rate vs. time
`mainInletMassFlow/`	Main-inlet mass flow rate vs. time
`wallH2Probes/`	H_2 at four wall probe locations

All files are plain-text columns (time + value) that can be plotted directly with `gnuplot`, Python, or imported into Excel.

Chapter 8

Quick-Start Summary

The complete workflow from a clean directory to a running simulation:

1. **Install dependencies:**

```
pip install numpy numpy-stl
```

2. **Generate STL geometry:**

```
python3 generateSTL.py
```

3. **Create background mesh:**

```
blockMesh
```

4. **Extract surface features:**

```
surfaceFeatureExtract
```

5. **Generate the final mesh:**

```
snappyHexMesh -overwrite
```

6. **Patch initial conditions:**

```
setFields
```

7. **Decompose for parallel run:**

```
decomposePar
```

8. **Run the simulation:**

```
mpirun -np 8 reactingFoam -parallel
```

9. **Reconstruct and post-process:**

```
reconstructPar
paraFoam
```

Or simply run everything with:

```
./Allrun
```

Checklist before running:

- ✓ STL files present in `constant/triSurface/`
- ✓ `thermophysicalProperties` species match `0/H2` and `0/CH4`
- ✓ Boundary patch names in all `0/` files match `snappyHexMeshDict` surface names
- ✓ `decomposeParDict` `numberOfSubdomains` matches `mpirun -np` count
- ✓ `controlDict` `endTime` and `deltaT` give desired physical time
- ✓ `fieldAverage1` `timeStart` set to 4.0 s (skip initial transient)